

TEMA 4 – Adaptadores y Listas

Objetivo del tema

En este tema aprenderás a utilizar el RecyclerView, un componente que permite mostrar listados personalizables en las pantallas de tus App. Para ello, necesitarás hacer uso de los Adapters.

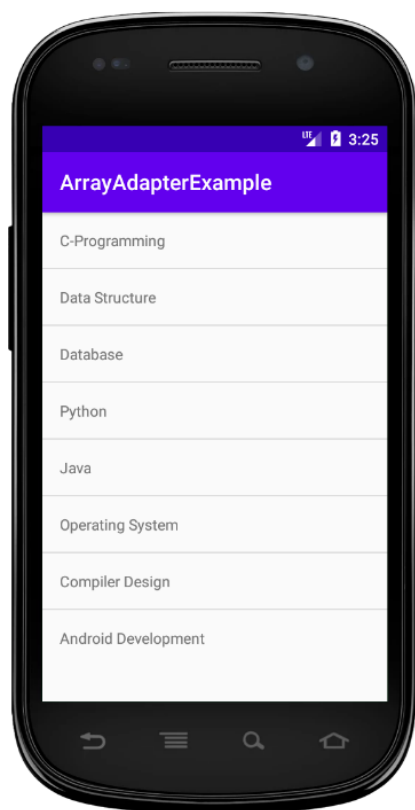
4.1 Adaptadores y Listas

Una **lista** es un *widget* muy utilizado que muestra una serie de elementos ordenados de forma vertical y con un scroll. Es muy posible que los hayas utilizado sin ni siquiera darte cuenta como, por ejemplo, cuando eliges una canción de una App de música. Son similares los JTable de Java.

Android ofrece dos componentes para trabajar con listas: el **ListView** y el **RecyclerView**. Ambos pueden ser incluido en un layout, normalmente ocupando casi toda la pantalla.

ListView existe desde la primera versión de Android, mientras que **RecyclerView** desde Android 5.0 (*Lollipop*). Se recomienda siempre usar siempre la **RecyclerView**, dado que está pensada para grandes listados y hace énfasis en la reutilización de vistas de los elementos que no se están mostrando en pantalla en ese momento, mejorando la eficiencia. Eso sí, programarla es un poco más complejo.

En estos apuntes nos centraremos solamente en el **RecyclerView**.



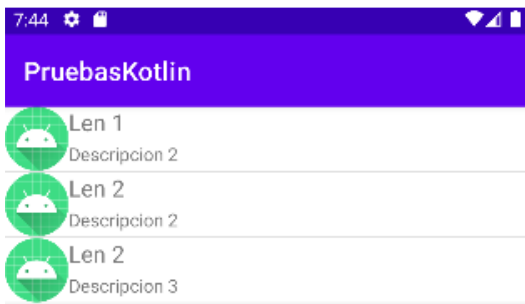
Por su parte, los **adaptadores (adapters)** son vitales para trabajar con listas, ya que vinculan cada elemento que queremos mostrar en la lista con cada uno de los ítems de la lista. Por explicarlo brevemente, si queremos mostrar en una lista el contenido de una lista de objetos Factura, tendremos que decirle exactamente cómo y en qué 'columna' tiene que poner el producto, el precio, la fecha, etc. de cada Facturas. De esto es de lo que se encarga un **adapter**, y lo hace de forma automática.

Los adaptadores NO son exclusivos de los **RecyclerView**. Componentes como **GridView** o el **Spinner** también los utilizan.

Todos los adapters de un **ListView** descienden de la clase BaseAdapter. Android dispone de dos, el **ArrayAdapter** y el **CursorAdapter**. El primero maneja datos provenientes de un ArrayList y el segundo de una Base de Datos. También puedes crearte tu propio adapter customizado, algo bastante habitual cuando quieres hacer listas que muestren fotos, botones, etc.

En cambio, no se ofrece ningún adaptador para **RecyclerView**. Todos los adaptadores deberán de ser customizados por el programador generando una clase adapter nueva que extienda de la clase RecyclerView.Adapter.

4.2 Adaptadores



Dado que siempre tendremos que hacer una adapter 'a mano' para cada **RecyclerView**, vamos a ver un ejemplo sencillo. Queremos conseguir algo como esto, una lista en la que cada elemento conste de un icono y dos textos que tienen que mostrarse de la siguiente forma:

La lista contendrá una serie de lenguajes, así que el icono, el nombre del idioma y su descripción deberán de ser acordes con cada uno de ellos.

Paso 1 – Comenzaremos creando la clase **Lenguajes** en Kotlin que define dichos atributos:

```
class Lenguajes (var nombre : String, var descripcion : String, var icono : Int)
```

Recuerda: en Kotlin puedes escribir el constructor en la cabecera de la clase y pasarle parámetros de golpe. Si no dices nada más, el compilador asume que las variables son atributos de clase, que automáticamente tienen los métodos get/set, y que lo que quieres hacer es un constructor que cargue dichos parámetros.

Paso 2 – Adapter: Lo siguiente es especificar el Adapter, al que llamaremos **LenguajesArrayAdapter**; y un *layout* nuevo que añadiremos en **/res** al que llamaremos **item_language**. Este nuevo layout nos servirá para especificar cómo tiene que verse cada uno de los ítems del RecyclerView.

Habitualmente, un layout para un adapter se crea del tipo **LinearLayout**, por sencillez, pero puedes utilizar el layout que quieras.

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <TextView
        android:id="@+id/itemTextView"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_gravity="center" />

</LinearLayout>
```

En el ejemplo que ves, el **LinearLayout** sólo dispone de un **TextView**: por lo tanto, el **RecyclerView** sólo mostrará un texto por cada elemento de la lista. Por tanto, deberás de modificar el layout añadiendo elementos nuevos para poder mostrar de la forma indicada los tres elementos de cada **Lenguaje** (icono, nombre y descripción).

Respecto al **LenguajesArrayAdapter**, nos quedaría algo similar a esto:

```
class LenguajesArrayAdapter (
    context: Context?,
    resource: Int,
    objects: List <Lenguajes>?
): ArrayAdapter<Lenguajes>(context!!, resource, objects!!) {

    override fun getView (position: Int, convertView: View?, parent: ViewGroup): View {
        val view : View = convertView?: LayoutInflater.from(this.context)
            .inflate(R.layout.item_language, parent, attachToRoot: false)

        val name = view.findViewById(R.id.nombre) as TextView
        val desc = view.findViewById(R.id.descripcion) as TextView
        val img = view.findViewById(R.id.icono) as ImageView

        getItem(position)?.let{ it: Lenguajes
            name.text = it.nombre
            desc.text = it.descripcion
            img.setImageResource(it.icono)
        }

        return view
    }
}
```

Paso 3 – Ya sólo nos queda vincular Adapter, Layout y la lista de los elementos a mostrar en nuestra Activity:

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

    val myList = findViewById<ListView>(R.id.myList)
    val lenguajes = mutableListOf<Lenguajes>()

    lenguajes.add (Lenguajes( nombre: "Len 1", descripcion: "Descripcion 2", R.mipmap.ic_launcher))
    lenguajes.add (Lenguajes( nombre: "Len 2", descripcion: "Descripcion 2", R.mipmap.ic_launcher))
    lenguajes.add (Lenguajes( nombre: "Len 2", descripcion: "Descripcion 3", R.mipmap.ic_launcher))

    val adapter = LenguajesArrayAdapter ( context: this, R.layout.item_language, lenguajes)
    myList.adapter = adapter
}
```

¿Pero cómo y por qué se carga el RecyclerView? Todo esto ocurre en tiempo de ejecución. Aunque en el ejemplo cargamos la lista mutable de lenguajes a mano, podría cargarse dinámicamente desde una base de datos, por ejemplo. Cada uno de esos lenguajes (objetos de tipo Lenguaje) tiene un icono, un nombre y una descripción. A continuación, le pasamos la lista al **LenguajesArrayAdapter** (adapter) y la referencia del layout que tiene que usar ‘de molde’: **item_language**. Y finalmente, le añadimos al **RecyclerView** (myList) el adapter.

Por tanto, en tiempo de ejecución lo que sucede es que, por cada uno de los elementos de la lista lenguajes, se va a hacer una llamada al adapter. Si te fijas, ese adapter internamente relaciona cada uno de los atributos de esa lista con un campo definido en el layout **item_language**. Visualmente el resultado final será una lista en la que cada elemento estará ‘maquetado’ como especifica el layout.

Ejercicio 11

Crea una App que muestre en un RecyclerView una serie de objetos Usuario. Cada objeto Usuario tiene un nombre, un apellido y una edad. Carga al menos cinco elementos en el RecyclerView.

Ejercicio 12

Modifica la App del ejercicio anterior para que incluya los siguientes botones:

- Nuevo Usuario: Añade un nuevo usuario al RecyclerView
- Borrar Usuario: Borra un usuario del RecyclerView por nombre
- Modificar usuario: Modifica la edad de un usuario por nombre del RecyclerView.

Ejercicio 13

Modifica la App del ejercicio anterior para que, en lugar de la edad, muestre un icono si el Usuario es mayor de 18 años y otro si es menor de edad.